

Reconfiguração Modular de Sistemas de Quóruns*

Eduardo Adilio Pelinson Alchieri¹, Alysson Neves Bessani²,
Fabiola Greve³, Joni da Silva Fraga⁴

¹CIC, Universidade de Brasília, Brasília - Brasil

²LaSIGE, Universidade de Lisboa, Lisboa - Portugal

³DCC, Universidade da Bahia, Bahia - Brasil

⁴DAS, Universidade Federal de Santa Catarina, Florianópolis - Brasil

Abstract. *Quorum systems are useful tools for implementing consistent and available storage in the presence of failures, especially in modern distributed systems such as clouds and MANETs. This paper proposes FREESTORE, a set of fault-tolerant protocols that emulates a register in dynamic asynchronous systems in which processes are able to join/leave the servers set by reconfigurations. The reconfiguration protocols presented are modular and loosely coupled with read/write protocols, making it easy to adapt static fault-tolerant registers to dynamic environments. By an analytical study, we show that FREESTORE is more efficient (in terms of communication steps) than previous solutions.*

Resumo. *Sistemas de quóruns são úteis na implementação consistente e confiável de sistemas de armazenamento de dados em presença de falhas, sobretudo em sistemas distribuídos modernos, tais como nuvens e MANETs. Este artigo propõe o FREESTORE, um conjunto de protocolos capazes de emular um registrador em sistemas dinâmicos assíncronos, onde processos podem entrar e sair do sistema, durante sua execução, através de reconfigurações. A reconfiguração proposta é modular e fracamente acoplada aos protocolos de leitura e escrita, facilitando a adaptação de registradores estáticos para funcionamento em ambientes dinâmicos. Uma análise detalhada mostra que essas características tornam o FREESTORE mais eficiente (em termos de passos de comunicação) do que outras soluções encontradas na literatura.*

1. Introdução

Os sistemas de quóruns [Gifford 1979] são abstrações fundamentais usadas para garantir consistência e disponibilidade de dados armazenados de forma replicada em um conjunto de servidores. Além de serem blocos básicos de construção para protocolos de sincronização (ex.: consenso), o grande atrativo destes sistemas está em seu poder de escalabilidade e balanceamento de carga, uma vez que as operações não precisam ser executadas por todos os servidores do sistema, mas apenas por um quórum dos mesmos. Atualmente, num contexto de computação em nuvem, quóruns são fundamentais para implementação das garantias de confiabilidade [Vogels 2009].

Sistemas de quóruns foram inicialmente estudados em ambientes estáticos, onde não é permitida a entrada e saída de servidores durante a execução do sistema [Gifford 1979, Attiya et al. 1995, Malkhi and Reiter 1998]. Esta abordagem não é adequada para sistemas que permanecerão em execução por um longo tempo, uma vez que,

*Este trabalho é parcialmente suportado pela Fundação de Ciência e Tecnologia (FCT) de Portugal através dos projetos SITAN (PTDC/EIA-EIA/113729/2009) e LaSIGE (PEst-OE/EEI/UI0408/2014).

dispondo de um quantidade suficiente de tempo, um adversário pode comprometer um número maior de servidores do que o tolerado e então quebrar as propriedades do sistema. Outra limitação é que estes protocolos não permitem que um administrador, em tempo de execução, adicione máquinas no sistema (para suportar um aumento na carga de processamento) ou troque máquinas antigas. Além disso, estes protocolos não são adequados para sistemas distribuídos modernos, desenvolvidos para redes dinâmicas, móveis ou auto-organizáveis (e.x., MANETs, P2P, Nuvens), onde, pela sua própria natureza, o conjunto de processos que compõem o sistema sofre modificações durante a execução.

O processo de modificar o conjunto de processos que compõem o sistema chama-se *reconfiguração*. Algumas propostas para sistemas de quóruns reconfiguráveis [Gilbert et al. 2010, Martin and Alvisi 2004] utilizam consenso [Lamport 1998] para garantir que todos os processos concordem com o conjunto de processos que fará parte do sistema. Entretanto, é conhecido que o consenso torna-se impossível num ambiente assíncrono sujeito a falhas de servidores e, para resolvê-lo, abstrações de sincronia precisam ser incorporadas ao sistema, como os detectores de falhas [Chandra and Toueg 1996]. Ocorre que a manutenção da consistência de um sistema de quóruns estático pode ser feita sem a necessidade de acordo e, portanto, num sistema totalmente assíncrono. Até pouco tempo, ainda não se sabia se seria possível implementar reconfigurações sem necessidade de acordo. O *DynaStore* [Aguilera et al. 2011] responde a essa questão apresentando um conjunto de algoritmos que implementam um sistema de quóruns dinâmico sem o auxílio de consenso. Outra proposta recente que segue esta abordagem, mas para sistemas Bizantinos, é o QUINCUNX [Alchieri et al. 2012].

Este artigo apresenta o FREESTORE, um conjunto de algoritmos que permite atualizar o conjunto de servidores do sistema durante sua execução e implementa um registrador atômico [Lamport 1986] tolerante a falhas por parada (*crash*) e livre de espera [Herlihy 1991]. Dois conjuntos de protocolos são propostos: (i) protocolos de leitura e escrita – que podem ser adaptados a partir de qualquer registrador estático (ex., o clássico ABD [Attiya et al. 1995]) e (ii) protocolos de reconfiguração – que implementam o processo de reconfiguração do conjunto de servidores presentes no sistema.

O FREESTORE melhora o estado da arte na área em pelo menos três aspectos: *modularidade*, *eficiência* e *simplicidade*. A modularidade é conseguida através da separação entre protocolos de reconfiguração e protocolos de leitura e escrita. Tal desacoplamento permite que o FREESTORE seja muito mais eficiente, em termos de passos de comunicação, do que outras propostas baseadas [Gilbert et al. 2010] ou não [Aguilera et al. 2011] em consenso, tanto nos protocolos de leitura e escrita quanto na reconfiguração. Finalmente, consideramos a estratégia empregada no protocolo de reconfiguração do FREESTORE mais simples de entender do que outras propostas, elucidando o processo de reconfiguração de sistemas de quóruns.

Resumidamente, as principais contribuições deste artigo são as seguintes: (i) apresentação do FREESTORE, um conjunto de algoritmos modular e eficiente para reconfiguração de sistemas de quóruns; (ii) apresentação do primeiro protocolo tolerante a falhas por parada para sistemas de armazenamento em ambientes dinâmicos no qual os protocolos de reconfiguração são desacoplados dos protocolos de leitura e escrita, facilitando a adaptação de registradores estáticos para funcionamento em ambientes dinâmicos; (iii) discussão sobre as principais diferenças entre utilizar ou não consenso em

reconfigurações; e (iv) apresentação de uma comparação detalhada do FREESTORE com outras soluções encontradas na literatura.

2. Definições Preliminares

2.1. Modelo de Sistema

Consideramos um sistema distribuído completamente conectado composto pelo conjunto universo de processos U , que é dividido em dois subconjuntos distintos: um conjunto infinito de servidores $\Pi = \{s_1, s_2, \dots\}$; e um conjunto infinito de clientes $C = \{c_1, c_2, \dots\}$. Clientes acessam o sistema de armazenamento implementado por um subconjunto dos servidores (uma *visão*) executando operações de leitura e escrita (R/W). Cada processo do sistema (cliente ou servidor) possui um identificador único e está sujeito a falhas por parada (*crash*). A chegada dos processos no sistema segue o modelo de chegadas infinitas com concorrência desconhecida mas finita [Aguilera 2004]. Além disso, consideramos que os processos estão conectados através de canais confiáveis, os quais são implementáveis através de SSL/TLS [Dierks and Allen 1999].

Consideramos um sistema distribuído assíncrono, onde não existem limites para o tempo de transmissão de mensagens ou processamentos locais nos processos. No entanto, cada servidor do sistema tem acesso a um relógio local usado para iniciar reconfigurações. Estes relógios não são sincronizados e não existe qualquer limite (*bounds*) para seus desvios (*drifts*), sendo portanto apenas contadores.

2.2. Dinamicidade e Propriedades do Sistema

Durante a execução, uma sequência de visões é instalada no sistema através de reconfigurações causadas por entradas e saídas de servidores. A seguir são apresentadas algumas definições e propriedades satisfeitas pelo FREESTORE.

Updates. Definimos um $update = \{+, -\} \times \Pi$, onde a tupla $\langle +, i \rangle / \langle -, i \rangle$ indica que o servidor i solicitou um *join/leave* do sistema. O processo de *reconfiguração* considera as *updates* solicitados para definir a nova composição do sistema.

Visões. Uma visão v é composta por um conjunto de *updates* (representados por $v.entries$) e seu *membership* (representado por $v.members$) é definido como $v.members = \{i \in \Pi: \langle +, i \rangle \in v.entries \wedge \langle -, i \rangle \notin v.entries\}$. Por simplicidade, usamos $i \in v$ para simbolizar $i \in v.members$. Note que um processo i pode entrar e sair do sistema uma única vez, mas esta condição pode ser relaxada na prática adicionando-se um número de época em cada *update*.

Dizemos que uma visão v é *instalada em um servidor correto* caso o mesmo considere v sua *visão atual* e passa a executar as operações de clientes nesta visão. Neste caso, dizemos que v foi *instalada no sistema*. Em qualquer tempo real t , definimos $V(t)$ como sendo a *visão mais atual* (veja definição abaixo) instalada no sistema. Consideramos que $V(t)$ permanece *ativa* desde o tempo de sua instalação até que todos os servidores corretos de outra visão mais atual $V(t')$, $t' > t$, instalem $V(t')$.

Comparando visões. Comparamos duas visões v_1 e v_2 através de suas *entries*. Usamos a notação $v_1 \subset v_2$ e $v_1 = v_2$ como abreviação de $v_1.entries \subset v_2.entries$ e $v_1.entries = v_2.entries$, respectivamente. Caso $v_1 \subset v_2$, então v_2 é *mais atual* do que v_1 .

Bootstrapping. Assumimos uma visão inicial não vazia $V(0)$ conhecida por todos os processos do sistema. Durante a inicialização do sistema, cada servidor $i \in V(0)$ recebe a visão inicial $v_0 = \{u_0\}$, onde $u_0 = \{\langle +, j \rangle: j \in V(0)\}$.

Visões vs. operações de R/W. Em qualquer tempo real t , operações de r/w são executadas apenas em $V(t)$. Quando um servidor i requisita um *join* no sistema, estas operações permanecem desabilitadas em i até a ocorrência de um evento *enable operations*. Após isso, i permanecerá apto a responder estas operações até que i solicite o *leave* do sistema, o que acontece com a ocorrência do evento *disable operations*.

Definição 1 (Propriedades do FREESTORE) As seguinte propriedades são satisfeitas:

- **Armazenamento (Segurança):** Os protocolos de r/w satisfazem as propriedades de segurança de um registrador de r/w atômico [Lamport 1986].
- **Armazenamento (Terminação):** Todas operações de r/w executadas por clientes corretos terminam.
- **Reconfiguração – Join (Segurança):** Se um servidor j instala a visão v tal que $i \in v$, então o servidor i executou a operação *join* ou é membro da visão inicial.
- **Reconfiguração – Leave (Segurança):** Se um servidor j instala a visão v tal que $i \notin v \wedge (\exists v' : i \in v' \wedge v' \subset v)$, então o servidor i executou a operação *leave*.
- **Reconfiguração – Join (Terminação):** O evento *enable operations* terminará por ocorrer em todo servidor correto que executa a operação *join*.
- **Reconfiguration – Leave (Terminação):** O evento *disable operations* terminará por ocorrer em todo servidor correto que executa a operação *leave*.

Para lidar com a dinamicidade do sistema, algumas suposições são necessárias. As suposições 1 e 2 são adaptações diretas dos resultados para sistemas de quóruns estáticos [Attiya et al. 1995] e devem ser satisfeitas enquanto a visão v estiver ativa. A Suposição 3 diz que um servidor correto que está deixando o sistema participará do protocolo de reconfiguração. Já a Suposição 4 garante que operações de r/w sempre terminam, i.e., garante que um cliente lento reiniciará sua operação, devido a reconfigurações, um número finito de vezes e, então, completará sua operação¹. Esta suposição é necessária em todas as propostas para sistemas de quóruns dinâmicos [Aguilera et al. 2011, Gilbert et al. 2010, Martin and Alvisi 2004].

Suposição 1 (Limiar de falhas) Para cada visão v , denotamos $v.f$ como sendo o número de falhas toleradas em v , $v.f \leq \lfloor \frac{|v.members|-1}{2} \rfloor$.

Suposição 2 (Tamanho dos quóruns) Para cada visão v , consideramos quóruns de tamanho $v.q = \lceil \frac{|v.members|+1}{2} \rceil$.

Suposição 3 (Saídas graciosas) Um servidor correto $i \in V(t)$ que requisita um *leave* no tempo t permanece no sistema até saber que uma visão mais atual $V(t')$, $t' > t$, $i \notin V(t')$ foi instalada no sistema.

Suposição 4 (Reconfigurações finitas) O número de updates requisitados em uma execução é finito.

3. Geradores de Sequências de Visões

Geradores de sequências de visões ($\mathcal{G}$) são módulos locais usados pelos servidores para gerar sequências de novas visões para reconfigurar o sistema. Este módulo captura os requisitos de sincronia necessários para a reconfiguração do sistema. Note que estes geradores geram sequências de visões (e não uma única visão), pois o principal desafio dos protocolos de reconfiguração é garantir a convergência para uma sequência de visões geradas durante a reconfiguração [Aguilera et al. 2010, Aguilera et al. 2011, Gilbert et al. 2010].

¹ Apenas o número de updates requisitados concorrentemente com uma operação de r/w deve ser finito.

Para cada visão v , cada servidor $i \in v$ associa um gerador $\mathcal{G}_i^v$ para gerar sequências de visões para atualizar o sistema. Um servidor i interage com o gerador através de duas primitivas: (1) $\mathcal{G}_i^v.gen_view(seq)$, chamada por i para propor uma nova sequência de visões seq para atualizar v ; e (2) $\mathcal{G}_i^v.new_view(seq)$, um *callback* invocado pelo gerador para informar i sobre uma nova sequência seq gerada. Como outros oráculos distribuídos (ex., detectores de falhas [Chandra and Toueg 1996]), os geradores podem implementar estas operações de diferentes formas, de acordo com os diferentes ambientes nos quais irão executar (ex., síncrono ou assíncrono). No entanto, neste trabalho introduzimos geradores que satisfazem as seguintes propriedades.

Definição 2 (GERADORES) Um gerador $\mathcal{G}_i^v$ (associado à visão v no processo i) satisfaz as seguintes propriedades:

- **Exatidão:** existem duas variantes de exatidão:
 - **Exatidão Forte:** se i receber o upcall $\mathcal{G}_i^v.new_view(seq_i)$, então todos os servidores $j \in v$ recebem upcalls $\mathcal{G}_j^v.new_view(seq_j)$ tal que $seq_i = seq_j$.
 - **Exatidão Fraca:** se i receber o upcall $\mathcal{G}_i^v.new_view(seq_i)$, então todos os servidores $j \in v$ recebem upcalls $\mathcal{G}_j^v.new_view(seq_j)$ tal que $seq_i \subseteq seq_j$ ou $seq_j \subseteq seq_i$.
- **Não trivialidade:** para todas upcalls $\mathcal{G}_i^v.new_view(seq_i)$, para qualquer visão $w \in seq_i$, temos que $v \subset w$.
- **Terminação:** para qualquer invocação $\mathcal{G}_i^v.gen_view(seq)$, existirá uma upcall $\mathcal{G}_i^v.new_view(seq_i)$, a menos que i falhe.

As propriedades de *Exatidão* e *Não trivialidade* estão relacionadas com a segurança, enquanto que *Terminação* está relacionada com a vivacidade do processo de geração de sequências de visões. Qualquer gerador deve atender as propriedades de Não trivialidade e Terminação para garantir que apenas visões atualizadas estarão nas sequências que sempre serão geradas. Usando as duas variantes de Exatidão definimos dois geradores: $\mathcal{P}$, gerador *perfeito* que satisfaz *Exatidão Forte*; e $\mathcal{L}$, gerador *vivo* que satisfaz apenas *Exatidão Fraca*. A implementação de $\mathcal{P}$ requer consenso, enquanto $\mathcal{L}$ pode ser implementado sem primitivas fortes de sincronização.

3.1. Gerador de Sequências de Visões Perfeito – $\mathcal{P}$

Geradores perfeitos garantem que uma única sequência de visões é gerada em todos os servidores que iniciaram o gerador para atualizar uma visão v . Nossa implementação de $\mathcal{P}$ (Algoritmo 1) é baseada no algoritmo de consenso Paxos [Lamport 1998] que requer um sistema parcialmente síncrono [Dwork et al. 1988]. Considerando o algoritmo do Paxos, qualquer servidor de v pode ser proponente (invocando $Paxos_v.propose(seq)$), mas todos os servidores de v são aceitantes e aprendizes (recebem um upcall $Paxos_v.learn(seq')$).

Algoritmo 1 $\mathcal{P}$ associado a v - servidor $i \in v$.

```

procedure  $\mathcal{G}_i^v.gen\_view(seq)$ 
1) if  $\forall w \in seq : v \subset w$  then                                     //somente visões atualizadas são propostas
2)    $Paxos_v.propose(seq)$                                            //inicializa um consenso entre os membros de  $v$ 
upon    $Paxos_v.learn(seq')$ 
3)    $\mathcal{G}_i^v.new\_view(seq')$                                            // ... informa a decisão para o processo.

```

3.2. Gerador de Sequências de Visões Vivo – $\mathcal{L}$

Nossa implementação para $\mathcal{L}$ (Algoritmo 2) não utiliza consenso, pois não é necessário que a mesma sequência seja gerada em todos os processos, sendo implementável em um sistema assíncrono. Desta forma, duas ou mais sequências podem ser geradas em diferentes servidores, mas a propriedade de intersecção dos quórums garante que uma sequência gerada estará contida em qualquer outra sequência gerada posteriormente. A ideia principal do algoritmo é fazer os processos atualizarem as suas propostas até que um quórum convirja para uma sequência contendo todas as propostas (ou uma combinação delas).

Algoritmo 2 $\mathcal{L}$ associado a v - servidor $i \in v$.

functions: Funções auxiliares e predicados
 $most_updated(seq) \equiv w: (w \in seq) \wedge (\nexists w' \in seq: w \subset w')$
variables: Conjuntos utilizados no protocolo
 $SEQ \leftarrow \emptyset$ // sequência proposta por i
 $LCSEQ \leftarrow \emptyset$ // última sequência convergida conhecida por i

procedure $\mathcal{G}_i^v.gen_view(seq)$
 1) **if** $SEQ = \emptyset \wedge \forall w \in seq: v \subset w$ **then**
 2) $SEQ \leftarrow seq$
 3) $\forall j \in v, send(SEQ-VIEW, SEQ)$ to j
upon receipt of $\langle SEQ-VIEW, seq \rangle$ from j
 4) **if** $\exists w \in seq: w \not\subseteq SEQ$ **then**
 5) **if** $\exists w, w': w \in seq \wedge w' \in SEQ \wedge w \not\subseteq w' \wedge w' \not\subseteq w$ **then**
 6) $w \leftarrow most_updated(SEQ)$
 7) $w' \leftarrow most_updated(seq)$
 8) $SEQ \leftarrow LCSEQ \cup \{w.entries \cup w'.entries\}$
 9) **else** // todas as visões podem ser agrupadas em uma sequência
 10) $SEQ \leftarrow SEQ \cup seq$
 11) $\forall k \in v, send(SEQ-VIEW, SEQ)$ to k
upon receipt of $\langle SEQ-VIEW, SEQ \rangle$ from $v.q$ servers in v
 12) $LCSEQ \leftarrow SEQ$
 13) $\forall k \in v, send(SEQ-CONV, SEQ)$ to k
upon receipt of $\langle SEQ-CONV, seq \rangle$ from $v.q$ servers in v
 14) $\mathcal{G}_i^v.new_view(seq)$

Neste algoritmo, cada servidor i pertencente a uma visão v utiliza: (i) uma função *most_updated* para extrair a visão mais atual de uma sequência (i.e., a visão que não está contida em nenhuma outra visão da sequência); (ii) uma variável *SEQ* para armazenar sua sequência proposta; e (iii) uma variável *LCSEQ* para armazenar a última sequência convergida em i . Quando o gerador é inicializado em um servidor $i \in v$, i verifica se já fez uma proposta ou se a sequência a ser proposta contém apenas visões atualizadas (linha 1), para então fazer sua proposta (linhas 2-3). No entanto, diferentes servidores de v podem propor diferentes sequências que devem ser combinadas em uma sequência composta por todas as propostas. Quando um servidor $i \in v$ recebe uma proposta de um servidor j , i verifica se a proposta contém algum visão que ainda não está contida em sua proposta (linha 4). Note que pode acontecer de i ainda não ter feito nenhuma proposta. Nestes casos, i atualiza sua proposta de acordo com dois casos excludentes:

CASO 1: [Existem *visões conflitantes* na sequência proposta por i e na sequência recebida (linhas 5-8)]. Para garantir que os servidores convirjam para uma mesma sequência e manter a relação (" $\subset$ ") entre as visões, i define sua proposta como sendo uma nova sequência contendo *LCSEQ* mais uma visão representando a união entre as propostas conflitantes.

CASO 2: [A sequência proposta por i (caso exista) e a sequência recebida *podem ser organizadas em uma nova sequência* (linhas 9-10)]. Então, a nova sequência a ser proposta por i é formada por todas as visões conhecidas por i .

Note que em ambos os casos é garantido que a nova proposta sempre contém

sequências previamente convergidas em pelo menos um quórum, i.e., sequências já geradas (ver abaixo). Quando i recebe a mesma proposta de um quórum de servidores em v , i converge para esta sequência atualizada seq (linhas 12-13). Quando i descobre que um quórum de processos convergiu para uma sequência seq , i gera seq (linha 14).

O ponto principal deste algoritmo é garantir que se um quórum de processos converge para uma sequência seq (i.e., executaram as linhas 12-13), seq é gerada (linha 14) e será uma subsequência de qualquer outra sequência gerada posteriormente, garantindo *Exatidão Fraca*. Isso é garantido pela propriedade de intersecção dos quóruns, i.e., pelo menos um servidor correto deve participar da geração de ambas sequências e aplicará as regras dos Casos 1 e 2 descritos acima.

Servidores de v podem gerar no máximo $|v.members| - v.q + 1$ sequências, tal que $seq_1 \subset seq_2 \subset \dots \subset seq_{(|v.members|-v.q)} \subset seq_{(|v.members|-v.q+1)}$: seq_1 é obtida pela união das propostas de um quórum de servidores em v e as outras $|v.members| - v.q$ sequências são construídas adicionando-se (cumulativamente) a seq_1 diferentes propostas de cada um dos $|v.members| - v.q$ servidores que não participaram da geração de seq_1 .

A propriedade de *Terminação* é garantida pelo fato de que (i) cada servidor faz apenas uma proposta inicial (linhas 1-3); (ii) os servidores atualizam suas propostas até que um quórum concorde com uma proposta; e (iii) sempre existe pelo menos um quórum de servidores corretos em v .

4. Protocolo de Reconfiguração do FREESTORE

Cada servidor que executa o algoritmo de reconfiguração do FREESTORE utiliza um gerador associado a sua visão atual cv para processar um lote de requisições de atualizações (*joins* e *leaves*) que atualizará o sistema de cv para uma nova visão w . Como estas requisições são processadas periodicamente em lotes, mais do que um *update* pode ser processado em cada execução do algoritmo. Além disso, estes protocolos também permitem que operações de *r/w* sejam executadas em paralelo com reconfigurações.

O Algoritmo 3 descreve como um servidor i executa reconfigurações. A seguir, descrevemos como os geradores são inicializados e discutimos o comportamento do algoritmo quando utiliza um gerador tanto da classe $\mathcal{L}$ (Seção 4.2) quanto da $\mathcal{P}$ (Seção 4.3).

4.1. Inicialização do Gerador de Sequências de Visões

O Algoritmo 3 (linhas 1-7) descreve como um servidor i processa requisições de *updates* e inicializa o gerador associado a sua visão atual cv . Um servidor j que deseja entrar no sistema primeiramente deve encontrar a visão atual cv e executar a operação *join* (linhas 1-2), enviando a tupla $\langle +, j \rangle$ para os membros de cv . Para sair do sistema o processo é similar (operação *leave* – linhas 3-4). Quando i recebe estas requisições de j , i verifica se a visão de j está desatualizada para então enviar sua cv para j (omitido do algoritmo por simplicidade). Caso contrário, i verifica se ainda não processou a requisição de j para então armazenar a requisição em RECV, que será processada na próxima reconfiguração. Finalmente, i envia um *acknowledgment* para j (linhas 5-6).

Por questões de desempenho, um relógio local é utilizado para execuções periódicas de reconfigurações (e não a cada *join/leave*). Um servidor $i \in cv$ inicia a reconfiguração de cv (i.e., inicia o gerador associado a cv) quando ocorrer o *timeout* de cv e existir *updates* pendentes (caso contrário, o *timeout* para cv é redefinido). O gerador é inicializado com uma sequência contendo uma visão que representa a cv de i atualizada

com os *updates* pendentes (linha 7). Este algoritmo garante que requisições recebidas por um quórum de servidores em *cv* serão executadas na reconfiguração seguinte.

4.2. Reconfiguração usando $\mathcal{L}$

Esta seção descreve o comportamento das reconfigurações quando o Algoritmo 3 é iniciado com um gerador $\mathcal{L}$ que pode gerar diferentes sequências de visões em diferentes servidores. Nestes casos, este algoritmo garante que uma única sequência de visões é instalada. A ideia principal é escolher apenas algumas visões vindas de $\mathcal{L}$ e definir uma sequência de visões a ser instalada até que uma única visão mais atual seja instalada.

Visão Geral. Depois que $\mathcal{L}$ gerou uma sequência, as visões desta sequência são ativadas, uma a uma, até que a visão mais atual da sequência seja instalada. Note que diferentes servidores podem gerar diferentes sequências, mas qualquer sequência gerada será uma subsequência de outra sequência gerada posteriormente.

Dada uma sequência $seq : w_1 \rightarrow \dots \rightarrow w_k \rightarrow w$ gerada para atualizar uma visão *v*, apenas *w* é instalada no sistema. As outras *k* visões ($w_1, \dots, w_k$) apenas são utilizadas como intermediárias e são chamadas de *visões auxiliares*. Caso um servidor conheça toda esta sequência, estas visões não são instaladas pois existe uma visão mais atual *w* para ser instalada. No entanto, é importante perceber que alguns servidores podem conhecer apenas uma subsequência de *seq* como $seq' : w_1 \rightarrow \dots \rightarrow w_j, j \leq k$, e instalar w_j . Neste caso, estes servidores acabarão atualizando suas visões de w_j para *w*.

Algoritmo 3 Reconfiguração - servidor *i*.

functions: Funções auxiliares e predicados
 $least_updated(seq) \equiv w: (w \in seq) \wedge (\nexists w' \in seq: w' \subset w)$
variables: Conjuntos e Gerador utilizado no protocolo
 $cv \leftarrow v_0$
 $RECV \leftarrow \emptyset$ // visão mais atual conhecida por *i*
// conjunto de *updates* recebidos

procedure *join()*
1) $\forall j \in cv, send(RECONFIG, \langle +, i \rangle, cv)$ to *j*
2) **wait** for $cv.q$ $\langle REC-CONFIRM \rangle$ replies from servers in *cv*

procedure *leave()*
3) $\forall j \in cv, send(RECONFIG, \langle -, i \rangle, cv)$ to *j*
4) **wait** for $cv.q$ $\langle REC-CONFIRM \rangle$ replies from servers in *cv*

upon receipt of $\langle RECONFIG, \langle *, j \rangle, cv \rangle$ from *j* $\wedge \langle *, j \rangle \notin cv$
5) $RECV \leftarrow RECV \cup \{ \langle *, j \rangle \}$
6) $send(REC-CONFIRM)$ to *j*

upon (*timeout* for *cv*) $\wedge (RECV \neq \emptyset)$
7) $\mathcal{G}_i^{cv}.gen_view(\{cv \cup RECV\})$

upon $\mathcal{G}_i^{ov}.new_view(ov, seq)$ //gerador associado a *ov* gerou uma nova sequência para atualizar *ov* (geralmente, *ov* = *cv*)
8) $w \leftarrow least_updated(seq)$ //a próxima visão na sequência *seq*
9) $R-multicast(ov \cup w, \langle INSTALL-SEQ, w, seq, ov \rangle)$

upon $R-delivery(ov \cup w, \langle INSTALL-SEQ, w, seq, ov \rangle)$
10) **if** *i* $\in ov$ **then** //i é membro da visão anterior
11) **if** *cv* $\subset w$ **then** stop the execution of *r/w* operations // enfileira estas operações enquanto transfere o estado
12) $\forall j \in w, send(\langle STATE-UPDATE, \langle val, ts \rangle, RECV \rangle)$ to *j* //i envia seu estado para os servidores de *w*
13) **if** *cv* $\subset w$ **then** //w é mais atual do que *cv* e o sistema reconfigurará de *cv* para *w*
14) **if** *i* $\in w$ **then** //i está na nova visão
15) **wait** for $\langle STATE-UPDATE, *, * \rangle$ messages from *ov.q* servers in *ov*
16) $\langle val, ts \rangle \leftarrow \langle val_h, ts_h \rangle$, pair with highest timestamp among the ones received above //i atualiza seu estado...
17) $RECV \leftarrow RECV \cup \{update\ requests\ from\ STATE-UPDATE\ messages\} \setminus w.entries$
18) $cv \leftarrow w$ //... e atualiza sua visão atual para *w*
19) **if** *i* $\notin ov$ **then** enable operations //i está entrando no sistema
20) $\forall j \in ov \setminus cv, send(\langle VIEW-UPDATED, cv \rangle)$ to *j* //informa os servidores de *ov* $\setminus cv$, os quais podem deixar o sistema
21) **if** $(\exists w' \in seq: cv \subset w')$ **then** //existem visões mais atuais do que *cv* em *seq*...
22) $seq' \leftarrow \{w' \in seq: cv \subset w'\}$ //... seleciona estas visões...
23) $\mathcal{G}_i^{cv}.gen_view(seq')$ //... e as propõe (volta para o Algoritmo 2)
24) **else**
25) resume the execution of *r/w* operations in *cv* = *w* and start a timer for *cv* //w é instalada
26) **else** //i está saindo do sistema
27) disable operations
28) **wait** for $\langle VIEW-UPDATED, w \rangle$ messages from *w.q* servers in *w* and then **halt** //i pode deixar o sistema

A propriedade de Exatidão Fraca garante que nenhuma operação de escrita executada em uma visão de seq' (caso instalada) é perdida. Para isso, é realizada uma “leitura em cadeia” onde os servidores de v transferem seus estados para os servidores de w_1 , os quais transferem seus estados para os servidores de w_2 e assim por diante, até que os servidores de w recebem o valor armazenado no sistema a partir dos servidores de w_k .

Reconfiguração. O Algoritmo 3 (linhas 8-28) apresenta a parte principal do protocolo de reconfiguração do FREESTORE. Este algoritmo utiliza a função *least_updated* para extrair a visão menos atualizada de uma sequência (i.e., a visão que está contida em todas as outras visões da sequência).

Quando o gerador associado a uma visão antiga ov gerar uma sequência de visões seq , a visão menos atualizada $w \in seq$ é a primeira a ser inicializada, sendo proposta na mensagem INSTALL-SEQ enviada para os servidores de $ov \cup w$ (linhas 8-9). Para garantir que todos receberão esta mensagem, usamos uma primitiva de *multicast* confiável [Hadzilacos and Toueg 1994]. Esta primitiva pode ser implementada eficientemente em sistemas assíncronos através da retransmissão de mensagens no receptor antes da entrega (i.e., em um passo de comunicação) [Hadzilacos and Toueg 1994].

Os últimos passos importantes do algoritmo são os seguintes. Primeiro, os servidores param de executar operações de r/w (linha 11), as quais serão executadas quando a visão mais atual da sequência é instalada (linha 25), o que garante que nenhuma operação de r/w é executada em visões auxiliares. Segundo, os servidores de ov enviam seus estados (geralmente o valor do registro e um *timestamp* [Attiya et al. 1995]) para os servidores de w (linha 12). Nestas mensagens também são enviados os conjuntos RECV e os servidores coletam estes *updates*, evitando a perda de *updates* requisitados durante reconfigurações. Terceiro, caso w não seja a visão mais atual de seq (w é uma visão auxiliar), uma sequência para atualizar w é proposta e os geradores associados a w são executados (linhas 21-23). Finalmente, um servidor que está deixando o sistema somente sai quando for garantido que a visão mais atual foi inicializada em um quórum de servidores (linha 28), garantindo a vivacidade do protocolo.

O FREESTORE satisfaz linearizabilidade [Herlihy and Wing 1990] devido a três pontos principais: (1) as visões instaladas no sistema formam uma única sequência; (2) durante a instalação de uma visão, o estado do registrador é atualizado através de uma “leitura em cadeia” nas visões anteriores da sequência, sendo que a última visão instalada sempre estará presente nesta sequência e nenhuma escrita é executada em visões auxiliares, garantindo que nenhuma escrita é perdida durante as reconfigurações; e (3) após uma visão v ser desinstalada (i.e., uma visão mais atual do que v ser instalada), operações de r/w não são mais executadas em v .

Exemplo de Execução. A Figura 1 ilustra um execução do protocolo de reconfiguração do FREESTORE usando $\mathcal{L}$ onde os servidores convergem para uma única visão, mesmo com propostas conflitantes. Considere a visão inicial $v_0 = \{1, 2, 3\}$. No exemplo, ① os servidores 1, 2, 3 receberam o *join* do processo 4 antes de inicializar seus geradores, enquanto que o *join* do processo 5 foi recebido apenas pelo processo 1. Consequentemente, os geradores associados com v_0 ($L(v_0)$) são inicializados nos servidores 2, 3 com uma sequência contendo a visão $v_1 = \{1, 2, 3, 4\}$ e no servidor 1 com a sequência contendo a visão $v_2 = \{1, 2, 3, 4, 5\}$ (linha 7). A propriedade de Exatidão Fraca de $L(v_0)$ garante que ② os servidores 2, 3 geram a sequência $seq_1^{v_0} = v_1$ e após, os servidores 1, 2, 3 geram a

sequência $seq_2^{v_0} = v_1 \rightarrow v_2$ ($seq_1^{v_0} \subset seq_2^{v_0}$).

A Figura mostra que ② os servidores 3, 4 $\in v_1$ receberam INSTALL-SEQ para $seq_1^{v_0}$ e ③ instalaram v_1 (linhas 11-31) antes de receber INSTALL-SEQ para $seq_2^{v_0}$. Adicionalmente, ③ os servidores 3, 4 inicializam seus geradores associados com v_1 ($L(v_1)$) e, ③ como receberam o *join* do servidor 6, $L(v_1)$ é inicializado com uma sequência contendo a visão $v_3 = \{1, 2, 3, 4, 6\}$ (linha 7). Por outro lado, os servidores 1, 2 recebem INSTALL-SEQ para $seq_2^{v_0}$ antes de receber estas mensagens para $seq_1^{v_0}$. Consequentemente, ③ v_1 será uma visão auxiliar nos servidores 1, 2 que iniciarão $L(v_1)$ com uma sequência contendo v_2 (linhas 22-23).

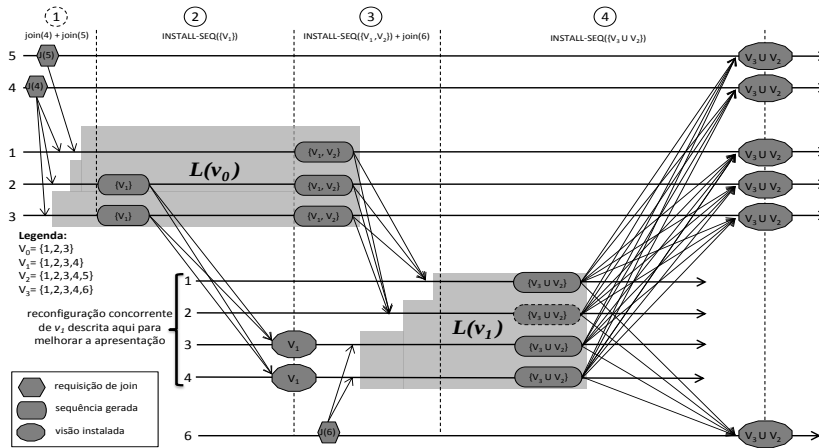


Figura 1. Convergência para uma única visão.

Neste cenário, os processos de v_1 estão com propostas conflitantes ($v_2 \not\subset v_3$ e $v_3 \not\subset v_2$) que devem ser organizadas em uma única sequência, por $L(v_1)$, para ser instalada após v_1 . Dependendo da ordem de recebimento das mensagens durante a execução de $L(v_1)$ (i.e., a composição do primeiro quórum de SEQ-VIEW recebido pelo processos de v_1), esta sequência poderá ser: (1) v_2 , (2) v_3 , (3) $v_2 \rightarrow \{v_3 \cup v_2\}$, (4) $v_3 \rightarrow \{v_3 \cup v_2\}$ ou (5) $\{v_3 \cup v_2\}$. Entretanto, a propriedade de Exatidão Fraca de $L(v_1)$ garante que os servidores não instalarão tanto v_3 quanto v_2 , i.e., existirá uma sequência única de visões instaladas. Em nosso exemplo, como tanto $\{1, 2\}$ quanto $\{3, 4\}$ não formam um quórum de v_1 , o conflito deve ser resolvido antes de qualquer sequência ser gerada. Consequentemente, a única sequência que poderá ser gerada é a (5). Assim, ④ $L(v_1)$ gera uma sequência $seq_1^{v_1} = \{v_3 \cup v_2\}$ e todos os servidores instalam a visão $v_3 \cup v_2$ (linhas 11-31).

Otimização. É possível reduzir um passo de comunicação no protocolo de reconfiguração (Algoritmo 3) quando utilizado com $\mathcal{L}$ (Algoritmo 2). Para isso, simplesmente basta usar as mensagens SEQ_CONV de $\mathcal{L}$ para informar os servidores que uma nova sequência deve ser instalada, eliminando as mensagens INSTALL_SEQ.

4.3. Reconfiguração usando $\mathcal{P}$

Caso um gerador desta classe seja usado com o protocolo de reconfiguração do FREE-STORE (Algoritmo 3), todos os geradores irão gerar a mesma sequência de visões (Exatidão Forte) que conterá uma única visão w , uma vez que todos propõem uma única visão (linha 7). Isso fará com que o sistema reconfigure diretamente da visão atual cv para w .

5. Protocolos de Leitura e Escrita

Esta seção discute como protocolos para sistemas de quóruns estáticos podem ser adaptados para sistemas dinâmicos através do FREE-STORE, o que é simplificado pelo fato

de as reconfigurações e as operações de r/w serem desacopladas. Basicamente, existem duas modificações principais. Primeiro, cada processo deve manipular uma variável cv para armazenar a visão mais atual conhecida e todas as mensagens do protocolo de r/w devem conter cv . Os servidores rejeitam qualquer operação enviada para uma visão antiga, respondendo com a visão atual. Os clientes então atualizam cv e reiniciam a fase da operação que está sendo executada (ver abaixo). Segundo, antes de acessar o sistema um cliente c deve obter a visão atual do sistema, o que pode ser implementado através de um serviço de diretório [Aguilera et al. 2010] onde os servidores armazenam as visões. Estes problemas são intrínsecos de protocolos para sistemas dinâmicos [Martin and Alvisi 2004, Aguilera et al. 2011, Gilbert et al. 2010].

Neste trabalho estendemos o clássico protocolo ABD [Attiya et al. 1995] para tolerar múltiplos escritores. A seguir, são apresentadas apenas os aspectos principais dos protocolos de r/w, os quais são executados em fases. Cada fase corresponde a um acesso a um quórum de servidores de cv . O protocolo de leitura funciona da seguinte forma:

- (*1ª Fase*) O leitor requisita um conjunto de tuplas $\langle val, ts \rangle$ de um quórum de servidores de cv (val é o valor armazenado pelos servidores e ts é o *timestamp* associado) e seleciona a tupla com o maior *timestamp* $\langle val_h, ts_h \rangle$. A operação termina e retorna val_h caso todas as tuplas coletadas sejam iguais, o que acontece em execuções sem concorrência com escritas ou falhas.
- (*2ª Fase*) Caso contrário, o cliente executa uma fase de *write-back* no sistema e aguarda por confirmações até ser garantido que um quórum de servidores de cv receberam $\langle val_h, ts_h \rangle$ e então retorna val_h .

O protocolo de escrita funciona de forma similar:

- (*1ª Fase*) O escritor obtém um conjunto de *timestamps* de um quórum de servidores de cv e escolhe o maior deles, ts_h . O *timestamp* de escrita ts é definido incrementando-se ts_h e concatenando-se o identificador do escritor nos *bits* menos significativos (para resolver conflitos) [Malkhi and Reiter 1998];
- (*2ª Fase*) O escritor envia a tupla $\langle val, ts \rangle$ para os servidores de cv , escrevendo val com *timestamp* ts , e aguarda por um quórum de confirmações.

O desacoplamento entre os protocolos de r/w e as reconfigurações possibilita a execução destas operações em paralelo com reconfigurações e simplifica a adaptação de qualquer implementação estática para ambientes dinâmicos.

6. Análise das Soluções para Reconfiguração de Sistemas de Quóruns

6.1. DynaStore vs. FREESTORE

Esta seção discute as principais diferenças entre as abordagens que não usam consenso.

Estratégia de Convergência. No DynaStore, reconfigurações geram um grafo de visões a partir do qual é possível identificar uma sequência de visões estabelecidas (instaladas – Figura 2(a)). Visões não estabelecidas funcionam como auxiliares, mas são acessadas nas operações de r/w. Para qualquer visão estabelecida v , o número máximo de visões geradas é da ordem de $|v|$, i.e., cada membro de v pode gerar uma nova visão e combinações também são possíveis. Por outro lado, no FREESTORE sequências geradas por diferentes geradores são organizadas em uma única sequência a ser instalada (Figura 2(b)). Para qualquer visão v , nossa implementação de $\mathcal{L}$ limita o número de sequências geradas em $|v| - v.q + 1$, compostas por no máximo $|v|$ visões.

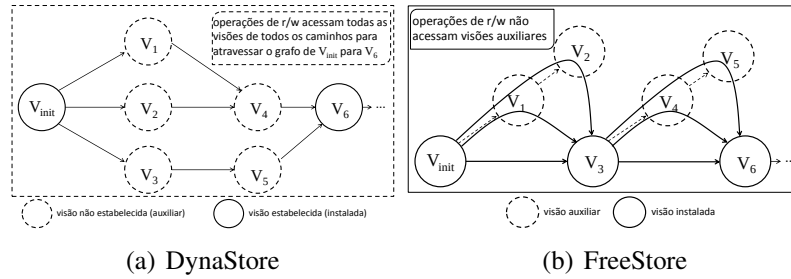


Figura 2. Dynastore vs. Freestore: convergência e operação.

Caso Normal de Execução. Para cada visão v (estabelecida ou não), o DynaStore associa um *weak snapshot object* wso_v que armazena as atualizações e é implementado por um conjunto S_v de $|v|$ (uma para cada membro de v) registradores SRMR (*single-writer multi-reader*), que são executados nos próprios servidores de v . Durante uma operação de r/w, wso_v deve ser acessado duas vezes para verificar se existem atualizações para v . Cada um destes acessos envolve duas leituras em cada registrador de S_v . Assim, para executar um operação de r/w, são necessários $4 * |v|$ (4 em sequência, $|v|$ em paralelo) acessos a um quórum (dois passos de comunicação). Por outro lado, o *overhead* introduzido pelo FREESTORE nas operações de r/w é a verificação da visão utilizada pelo cliente, i.e., verificar se esta visão está atualizada através da comparação de dois conjuntos (Seção 2.2).

Concorrência entre R/W e Reconfigurações. O DynaStore gera um grafo de visões para reconfigurar o sistema e uma operação de r/w deve percorrer este grafo para encontrar a visão mais atual. Para isso, cada aresta do grafo deve ser acessada. Para cada visão v (estabelecida ou não), é necessário acessar seu *weak snapshot object*, o que demanda $2 * |v|$ (2 em sequência, $|v|$ em paralelo) acessos a um quórum. Além disso, enquanto o sistema está convergindo para alguma visão, as operações de r/w não terminam. Por outro lado, no FREESTORE as operações de r/w não acessam visões auxiliares. Além disso, enquanto o sistema está convergindo, as operações de r/w são executadas concorrentemente na última visão instalada v . No entanto, após uma sequência ser obtida para atualização de v , as operações de r/w apenas terminam após a instalação da última visão desta sequência.

Passos de Comunicação para R/W. A Tabela 1 analisa o número de passos de comunicação necessários para executar uma operação de r/w. Para um cliente c que possui uma visão atualizada v , cada operação de r/w executada no DynaStore necessita dos seguintes passos: 2 acessos ao wso_v , resultando em 8 passos de comunicação para executar 4 leituras em cada registrador de S_v , verificando se v está atualizada; e 4 passos de comunicação para ler/escrever o valor de/para v . No FreeStore, uma operação de r/w necessita dos mesmos passos de comunicação usados no protocolo de r/w base (Seção 5).

A Tabela 1 também mostra os resultados para quando c está com a visão v desatualizada. Neste caso, o DynaStore precisa de 6 passos de comunicação para acessar cada aresta do grafo de visões. Para simplificar esta análise, consideramos que somente uma aresta deve ser acessada para encontrar a visão atualizada w (melhor caso). Assim, c necessita de 6 passos de comunicação para encontrar w , 12 passos de comunicação para executar a operação de r/w em w e 1 passo de comunicação para notificar os processos sobre a nova visão. No FREESTORE, para qualquer caso, 2 passos adicionais são necessários para receber w a partir dos servidores de v .

	DynaStore	FreeStore	Soluções Livres de Consenso	Melhor Caso	Pior Caso
	Visão Atualizada		DynaStore	23	$18 * v + 5$
Leitura	12	2/4	FREESTORE com $\mathcal{L}$	4	$7 * v - 2 * v.q - 1$
Escrita	12	4	FREESTORE com $\mathcal{L}$ (otimizado)	3	$5 * v - v.q - 1$
	Visão Desatualizada		Soluções Baseadas em Consenso	Melhor Caso	Pior Caso
Leitura	19	4/6	RAMBO	7	7
Escrita	19	6	FREESTORE com $\mathcal{P}$	5	5

Tabela 1. Custos das operações de leitura e escrita (esquerda) e reconfigurações (direita).

6.2. Reconfiguração Livre de Consenso vs. Baseada em Consenso.

Esta seção analisa os custos envolvidos na reconfiguração de uma visão v . Os resultados para as soluções livres de consenso (DynaStore [Aguilera et al. 2011] e FREESTORE com $\mathcal{L}$) e para duas solução que utilizam consenso para reconfigurar o sistema (RAMBO [Gilbert et al. 2003] e FREESTORE com $\mathcal{P}$) são apresentados na Tabela 1. São analisados os custos para o melhor caso, quando todos os processos propõem os mesmos *updates*, e para o pior caso, quando cada processo propõe diferentes *updates* para reconfigurar o sistema. Na análise das soluções livres de consenso, consideramos que nenhuma reconfiguração é inicializada paralelamente por processos de outras visões. Finalmente, consideramos uma execução síncrona do protocolo de consenso Paxos [Lamport 1998] para as soluções baseadas em consenso, o que requer três passos de comunicação.

Os custos para o melhor caso são definidos como segue: (i) DynaStore - somente um processo de v escreve um *update* no *weak snapshot object* associado a v (wso_v), desta forma são necessários 4 passos para escrever em wso_v , 6 passos para ler o valor do registrador de v , 6 passos para atravessar o grafo de visões e alcançar a nova visão w , 6 passos para escrever o valor do registrador em w e 1 passo para notificar os outros processos sobre w , totalizando 23 passos; (ii) FREESTORE com $\mathcal{L}$ - são necessários 4 passos, i.e., 1 SEQ_VIEW + 1 SEQ_CONV + 1 INSTALL_SEQ + 1 STATE_UPDATE; (iii) FREESTORE com $\mathcal{L}$ (otimizado) - como o anterior, mas as mensagens INSTALL_SEQ não são mais necessárias; (iv) RAMBO - 3 passos para executar um consenso, 2 passos para ler o valor da configuração (visão) antiga e 2 passos para escrever o valor na nova configuração (atualizando-a), totalizando 7 passos; e (v) FreeStore com $\mathcal{P}$ - 3 passos para executar um consenso mais 1 INSTALL_SEQ e 1 STATE_UPDATE, totalizando 5 passos.

Os custos para o pior caso são definidos como segue: (i) DynaStore - os processos de v escrevem seus *updates* nos *weak snapshot objects* formando um grafo com um único caminho de v para a visão mais atualizada composta pelo união de todos os *updates* (somente no final deste processamento uma operação de r/w termina), este caminho contém $|v|$ visões e, para atravessá-lo, um processo gasta 18 passos para cada visão, 4 passos para escrever seus *updates* e 1 passo para notificar os processos sobre uma visão mais atual, totalizando $18 * |v| + 5$ passos; (ii) FreeStore com $\mathcal{L}$ - os geradores associados com v geram no máximo $|v| - v.q + 1$ sequências contendo no máximo $|v|$ visões, para isso são necessárias $|v|$ SEQ_VIEW, $|v| - v.q + 1$ SEQ_CONV, $|v| - v.q + 1$ INSTALL_SEQ e 1 STATE_UPDATE, já os geradores associados com as outras $|v| - 1$ visões recebem estas sequências e terminam em 4 passos (um para cada mensagem acima citada) demandando mais $4(|v| - 1)$ passos, totalizando $7 * |v| - 2 * v.q - 1$ passos; (iii) FreeStore com $\mathcal{L}$ (otimizado) - como o anterior, mas as mensagens INSTALL_SEQ não são mais necessárias. As soluções baseadas em consenso apresentam os mesmos custos para ambos os cenários.

7. Conclusões

Este artigo apresentou uma nova abordagem para reconfigurar sistemas de quóruns, a qual evidencia as diferenças entre o uso ou não de consenso. O principal resultado é um protocolo mais eficiente (em termos de passos de comunicação tanto para operações de r/w quanto para reconfigurações) do que outras soluções encontradas na literatura e que desacopla as operações de r/w das reconfigurações. Outro resultado interessante é que no melhor caso, reconfigurações que não utilizam consenso são mais baratas do que as que utilizam este protocolo de sincronização. Acreditamos que explorar diferentes implementações de geradores (e diferentes classes) é um importante caminho para trabalhos futuros que desejam investigar o problema da reconfiguração.

Referências

- Aguilera, M. (2004). A pleasant stroll through the land of infinitely many creatures. *SIGACT News*, 35(2).
- Aguilera, M. K., Keidar, I., Malkhi, D., Martin, J.-P., and Shraer, A. (2010). Reconfiguring replicated atomic storage: A tutorial. *Bulletin of the EATCS: The Distributed Computing Column*.
- Aguilera, M. K., Keidar, I., and Malkhi, A. Shraer, D. (2011). Dynamic atomic storage without consensus. *Journal of the ACM*, 58:7:1–7:32.
- Alchieri, E., Bessani, A., Fraga, J., and Greve, F. (2012). Memória compartilhada em sistemas bizantinos dinâmicos. In *Anais XIII Workshop de Teste e Tolerância a Falhas*.
- Attiya, H., Bar-Noy, A., and Dolev, D. (1995). Sharing memory robustly in message-passing systems. *Journal of the ACM*, 42(1):124–142.
- Chandra, T. D. and Toueg, S. (1996). Unreliable failure detectors for reliable distributed systems. *Journal of the ACM*, 43(2):225–267.
- Dierks, T. and Allen, C. (1999). The TLS Protocol Version 1.0 (RFC 2246). IETF Request For Comments.
- Dwork, C., Lynch, N. A., and Stockmeyer, L. (1988). Consensus in the presence of partial synchrony. *Journal of ACM*, 35(2):288–322.
- Gifford, D. (1979). Weighted voting for replicated data. In *7th ACM Symposium on Operating Systems Principles*.
- Gilbert, S., Lynch, N., and Shvartsman, A. (2003). Rambo ii: Rapidly reconfigurable atomic memory for dynamic networks. *Proceedings of the 33th International Conference on Dependable Systems and Networks*.
- Gilbert, S., Lynch, N., and Shvartsman, A. (2010). Rambo: A robust, reconfigurable atomic memory service for dynamic networks. *Distributed Computing*, 23(4).
- Hadzilacos, V. and Toueg, S. (1994). A modular approach to the specification and implementation of fault-tolerant broadcasts. Technical report, Dep. of Computer Science, Cornell University, New York - USA.
- Herlihy, M. (1991). Wait-free synchronization. *ACM Transactions on Programming Languages and Systems*, 13(1).
- Herlihy, M. and Wing, J. M. (1990). Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 12(3).
- Lamport, L. (1986). On interprocess communication (part II). *Distributed Computing*, 1(1):203–213.
- Lamport, L. (1998). The part-time parliament. *ACM Transactions Computer Systems*, 16(2):133–169.
- Malkhi, D. and Reiter, M. (1998). Secure and scalable replication in Phalanx. In *17th Symposium on Reliable Distributed Systems*, pages 51–60.
- Martin, J.-P. and Alvisi, L. (2004). A Framework for Dynamic Byzantine Storage. In *34th International Conference on Dependable Systems and Networks*.
- Vogels, W. (2009). Eventually consistent. *Commun. ACM*, 52(1):40–44.